نسخه‌ی A · PLAN

🗺️ PLAN.md — نقشه‌ی راه ۳ ساعته · نسخه‌ی A: MiniBank

این فایل را در **ریشه‌ی ریپازیتوری** خودت بگذار. بخش «تصمیم‌های من» را در مرحله‌ی ۰ پر کن و در پایان جدول خودارزیابی را کامل کن.

🧭 چطور از این نقشه استفاده کنی

هر مرحله این بخش‌ها را دارد:

نشانه	معنی
⏱️	زمان پیشنهادی (ساعت آزمون، از ۰:۰۰ تا ۳:۰۰)
🎯	هدف مرحله
✅	کارهایی که باید انجام بدهی
🏁	«تمام است وقتی...»: چطور بفهمی مرحله واقعاً تمام شده
💯	امتیازها؛ مدرس هم دقیقاً با همین چک‌باکس‌ها نمره می‌دهد
💡	نکته‌ها و راهنمایی
🤖	تله‌های هوش مصنوعی: اشتباه‌هایی که AI زیاد می‌کند و امتیاز تو را می‌گیرد
🚩	نقطه‌ی بازرسی: کامیت و ادغام (merge)

چهار قانون طلایی:

۱. **اول کار کند، بعد زیبا شود.** نسخه‌ی ساده‌ای که کار می‌کند از نسخه‌ی کامل نیمه‌کاره بهتر است.

۲. **گیر نکن.** امتیاز جزئی همه‌جا هست. اگر بیش از ۱۰ دقیقه روی یک مشکل ماندی، آن را در README یادداشت کن و برو مرحله‌ی بعد.

۳. **سراغ امتیاز اضافه فقط وقتی برو که** همه‌ی مراحل اصلی تمام شده باشد. (این یک توصیه است؛ امتیاز اضافه در هر حال شمرده می‌شود و در رسیدن به نمره‌ی قبولی هم حساب است.)

۴. **هر کدی که AI به تو داد، اجرا کن و بفهم.** سؤال‌های دفاع از همین کدها پرسیده می‌شود.

⏱️ جدول زمان‌بندی

زمان	مرحله	امتیاز	شاخه (branch)
۰:۰۰ تا ۰:۱۰	S0 شروع و «تصمیم‌های من»	۵	`main`
۰:۱۰ تا ۱:۰۵	S1 برنامه‌ی Go	۲۵	`feat/api`
۱:۰۵ تا ۱:۲۰	S2 تست‌ها	۸	`test/unit`

زمان	مرحله	امتیاز	شاخه (branch)
۱:۲۰ تا ۱:۳۵	S3 لینوکس و Bash	۷	`chore/scripts`
۱:۳۵ تا ۱:۵۵	S4 Docker	۹	`build/docker`
۱:۵۵ تا ۲:۱۰	S5 Docker Compose	۷	`build/compose`
۲:۱۰ تا ۲:۲۵	S6 CI با GitHub Actions	۶	`ci/github-actions`
۲:۲۵ تا ۲:۵۰	S7 Kubernetes	۱۰	`deploy/k8s`
۲:۵۰ تا ۳:۰۰	S8 مستندات و تحویل	۳	`docs/readme`
	جمع ساخت	**۸۰**	

مرحله‌ی ۰ — شروع و «تصمیم‌های من» (روش Q)

⏱️ ۰:۰۰ تا ۰:۱۰ · 💯 ۵ امتیاز · 🌿 `main`

🎯 قبل از نوشتن حتی یک خط کد، **مثل یک مهندس فکر کن.** روش Q چهار قدم دارد:

تحلیل ← سؤال ← توافق ← ساخت

۱. **تحلیل:** README را کامل بخوان و دقیق ببین چه چیزی خواسته شده.

۲. **سؤال:** از خودت بپرس کجاها مطمئن نیستی. (اینجا مشتری خودت هستی.)

۳. **توافق:** جواب‌ها را در بخش «تصمیم‌های من» پایین همین مرحله بنویس. این قرارداد تو با خودت است.

۴. **ساخت:** حالا کدنویسی را شروع کن.

✅ **کارها:**

- [ ] پوشه‌ی پروژه را بساز، بعد `git init` و `go mod init minibank`
- [ ] یک `.gitignore` بساز (باینری‌ها، `.env`، `*.out`، `coverage*`). برای باینری بنویس `/minibank` (با `/` در ابتدا) یا `bin/`؛ اگر فقط `minibank` بنویسی، Git پوشه‌ی `cmd/minibank/` را هم نادیده می‌گیرد و `main.go` کامیت نمی‌شود.
- [ ] ریپازیتوری را در GitHub بساز و `git remote add origin …`
- [ ] همین فایل PLAN.md را در ریشه‌ی پروژه بگذار و بخش «تصمیم‌های من» را پر کن
- [ ] کامیت `docs: my decisions` و `git push`، **قبل از ۰:۱۵**
- [ ] آدرس ریپازیتوری را تا ۰:۱۵ به روشی که مدرس اعلام می‌کند تحویل بده. اگر ریپازیتوری خصوصی است، حساب GitHub مدرس (نامش روی تخته است) را از Settings → Collaborators دعوت کن.

💯 **امتیازها:**

- [ ] **(۲)** بخش «تصمیم‌های من» کامل و مشخص (نه کلی‌گویی) پر شده و تا ۱۵ دقیقه بعد از شروع آزمون روی GitHub **push** شده است. همان نسخه‌ی pushشده نمره می‌گیرد؛ اگر بعداً تصمیمی را عوض کردی، زیر یک عنوان جدید «تغییرات بعد از شروع» بنویس.
- [ ] **(۱)** `go.mod` و `.gitignore` درست هستند و ریپازیتوری روی GitHub push شده است

☐ **(۲)** جریان شاخه‌ای: تا پایان آزمون دست‌کم **۴ شاخه‌ی مرحله‌ای** با **کامیت merge** در `main` ادغام شده باشد (هر کدام نیم امتیاز). پس همیشه `git merge --no-ff` بزن: merge معمولی اینجا fast-forward می‌شود و هیچ کامیت mergeی نمی‌سازد. در GitHub هم فقط دکمه‌ی **Create a merge commit** (نه Squash و نه Rebase). در `git log --graph` باید شاخه‌ها جدا دیده شوند.

💡 **جریان کار برای هر مرحله:** شاخه‌ی جدید بساز، چند کامیت کوچک بزن، بعد با `--no-ff` در `main` ادغام و push کن:

```
git switch -c feat/api
git add -A && git commit -m "feat: ..."
git switch main
git merge --no-ff feat/api -m "merge: feat/api"
git push origin main
```

✍️ تصمیم‌های من (My decisions)

> هر جواب ۲ تا ۵ خط باشد. **مشخص بنویس.** جمله‌ای مثل «از بهترین روش استفاده می‌کنم» جواب حساب نمی‌شود.
> خط «✍️ اینجا بنویس...» را **پاک کن** و جوابت را جایش بنویس (نه به‌صورت نقل‌قول با `>`). عنوان‌ها را عوض نکن، به‌خصوص «تصمیم‌های من (My decisions)» و کلمه‌های انگلیسی WHAT، STRUCTURE، DATA، CONCURRENCY، SECURITY، OUT OF SCOPE، RISKS و AI PLAN؛ تصحیح‌گر جواب‌ها را با همین عنوان‌ها پیدا می‌کند.

۱. WHAT — چه چیزی می‌سازم و برای چه کسی؟ (دو جمله)

✍️ *اینجا بنویس...*

۲. STRUCTURE — درخت پوشه‌ها و پکیج‌هایم، و این‌که هر پکیج چه مسئولیتی دارد

✍️ *اینجا بنویس...*

۳. DATA — جدول‌های PostgreSQL من (ستون‌ها، نوع‌ها و محدودیت‌ها)

✍️ *اینجا بنویس...*

۴. CONCURRENCY — چطور انتقال پول را اتمیک و در برابر درخواست‌های هم‌زمان امن می‌کنم؟ (هم در store درون‌حافظه‌ای و هم در PostgreSQL)

✍️ *اینجا بنویس...*

۵. SECURITY — سه اقدام امنیتی مشخص (مثلاً درباره‌ی SQL، رمزها، کانتینر یا ورودی کاربر)

✍️ *اینجا بنویس...*

۶. OUT OF SCOPE — امروز چه چیزهایی را عمداً نمی‌سازم؟

✍️ *اینجا بنویس...*

۷. RISKS — کدام بخش احتمالاً زمان‌بندی‌ام را خراب می‌کند و نقشه‌ی جایگزینم چیست؟

✍️ *اینجا بنویس...*

۸. AI PLAN — از هوش مصنوعی برای چه کارهایی استفاده می‌کنم و خروجی‌اش را چطور بررسی می‌کنم؟

✍️ *اینجا بنویس...*

🚩 **نقطه‌ی بازرسی ۰:** کامیت `docs: my decisions` روی `main` و push.

مرحله‌ی ۱ — برنامه‌ی Go

⏱️ ۰:۱۰ تا ۱:۰۵ · 💯 ۲۵ امتیاز · 🌿 `feat/api`

🎯 API کامل طبق قرارداد README؛ اول با store درون‌حافظه‌ای و بعد با PostgreSQL.

✅ **ترتیب پیشنهادی** (بعد از هر قدم اجرا و امتحان کن):

۱. **دامنه (حدود ۱۰ دقیقه)** — در `internal/bank` : typeهای `Account` و `Transaction` با تگ‌های JSON دقیق، خطاهای ثابت (sentinel)، و تابع‌هایی برای اعتبارسنجی `owner` و `amount` .

۲. **store درون‌حافظه‌ای (حدود ۱۰ دقیقه)** — در `internal/store/memory.go` : یک map به‌همراه یک `sync.Mutex` . تابع `Transfer` باید **فقط یک بار** قفل بگیرد و هر دو حساب را زیر همان قفل تغییر دهد.

۳. **لایه‌ی HTTP (حدود ۱۵ دقیقه)** — در `internal/api` : مسیرها با الگوهای Go 1.22، تابع‌های کمکی `writeJSON` و `writeError` ، و **یک تابع واحد** که خطا را با `errors.Is` به status code تبدیل کند.

۴. **main (حدود ۵ دقیقه)** — خواندن env، انتخاب store، ساخت `http.Server` ، middleware لاگ و recovery، و خاموشی مرتب. حالا با `STORE=memory go run ./cmd/minibank` نمونه‌ی curl در README را امتحان کن.

۵. **PostgreSQL (حدود ۱۵ دقیقه)** — در `internal/store/postgres.go` : ساخت جدول‌ها هنگام شروع برنامه (`CREATE TABLE IF NOT EXISTS`)، کوئری‌های پارامتری، و انتقال پول داخل یک تراکنش SQL.

💡 **یک PostgreSQL سریع برای توسعه:**

```
docker run -d --name pg -p 5432:5432 \
  -e POSTGRES_USER=minibank -e POSTGRES_PASSWORD=minibank -e POSTGRES_DB=minibank \
  postgres:16-alpine

export STORE=postgres
export DATABASE_URL='postgres://minibank:minibank@localhost:5432/minibank?sslmode=disable'
go run ./cmd/minibank
```

🏁 **تمام است وقتی:** همه‌ی ۹ endpoint هم با `STORE=memory` و هم با `STORE=postgres` درست جواب می‌دهند، و با Ctrl+C برنامه پیام خاموشی مرتب را چاپ می‌کند.

💯 **امتیازها:**

- ☐ **(۸)** قرارداد API — با تصحیح‌گر خودکار: همه‌ی endpointها، status codeها، نام فیلدها و ترتیب خطاها
- ☐ **(۳)** طوفان هم‌زمانی — با تصحیح‌گر خودکار: ۱۰۰ واریز هم‌زمان و ۵۰ انتقال هم‌زمان روی **PostgreSQL**؛ پول باید دقیقاً حفظ شود و هیچ پاسخ `500` ی نیاید
- ☐ **(۳)** دامنه و اعتبارسنجی: پول از نوع `int64` ، شمارش rune برای `owner` ، بازه‌ی مجاز `amount` ، خطاهای ثابت همراه با `errors.Is` ، و یک تابع واحد برای نگاشت خطا به status
- ☐ **(۴)** اینترفیس `Store` با دو پیاده‌سازی (memory با Mutex، و postgres) که با متغیر `STORE` انتخاب می‌شوند

- [ ] **(۳)** درستی PostgreSQL: ساخت schema هنگام شروع با ستون‌های `BIGINT` و `CHECK (balance >= 0)`، **فقط** کوئری‌های پارامتری (`$1` و `$2`)، انتقال و ردیف تراکنشش داخل **یک** تراکنش SQL، و بدون الگوی «اول بخوان، بعد بررسی کن، بعد بنویس» **بدون قفل** (یا `UPDATE` شرطی با `AND balance >= $1`، یا `SELECT … FOR UPDATE` داخل همان تراکنش)
- [ ] **(۱)** پیکربندی از env با مقدارهای پیش‌فرض؛ اگر `DATABASE_URL` نبود، خروج فوری با پیام واضح
- [ ] **(۱)** `/healthz` به دیتابیس دست نمی‌زند؛ `/readyz` دیتابیس را ping می‌کند (حداکثر ۲ ثانیه) و اگر قطع بود `503` برمی‌گرداند
- [ ] **(۱)** middleware: لاگ JSON برای هر درخواست با `log/slog`، و recovery از panic که `500` و `{"error":"internal error"}` برگرداند
- [ ] **(۱)** خاموشی مرتب با `SIGINT` / `SIGTERM`: فراخوانی `http.Server.Shutdown` با timeout، بستن اتصال دیتابیس و نوشتن پیام در لاگ

💡 **نکته‌ها:**

- **برداشت امن بدون race در PostgreSQL** با یک دستور شرطی انجام می‌شود:

```
UPDATE accounts SET balance = balance - $1
WHERE id = $2 AND balance >= $1
RETURNING id, owner, balance, created_at;
```

اگر هیچ ردیفی برنگشت، یا حساب وجود ندارد (`404`) یا موجودی کافی نیست (`409`). با یک `SELECT` جداگانه تشخیص بده کدام است.

- **انتقال** فرق دارد، چون ترتیب خطاها مهم است (`404` قبل از `409`، حتی وقتی مبدأ پول ندارد و مقصد اصلاً وجود ندارد). داخل یک تراکنش (`BEGIN … COMMIT`): اول **هر دو** ردیف را با `SELECT … FOR UPDATE` قفل کن و اگر یکی نبود `404` بده؛ بعد موجودی مبدأ را بررسی کن (`409`)؛ بعد دو `UPDATE` و ثبت ردیف تراکنش. (اگر وجود مقصد را بررسی نکنی، `UPDATE` روی حساب ناموجود هیچ خطایی نمی‌دهد و پول گم می‌شود.)
- یک slice خالی (`nil`) در JSON به `null` تبدیل می‌شود! برای لیست خالی از `[]bank.Account{}` یا `make` استفاده کن تا خروجی `[]` باشد.
- اگر عدد `10.5` را در فیلدی از نوع `int64` کنی، `json.Decoder` خطا می‌دهد. همین خطا را به `400` تبدیل کن.
- شناسه‌ی مسیر را با `r.PathValue("id")` بگیر و با `strconv.ParseInt` به عدد تبدیل کن.
- در store درون‌حافظه‌ای، اگر `Transfer` تابع‌های `Withdraw` و `Deposit` را صدا بزند و هر کدام جداگانه قفل بگیرند، یا اتمیک نیست یا (با Mutex غیر بازگشتی Go) در بن‌بست گیر می‌کند. فقط یک بار قفل بگیر.

🤖 **تله‌های هوش مصنوعی:**

- پول را `float64` در نظر می‌گیرد.
- در PostgreSQL اول `SELECT balance` (بدون `FOR UPDATE`) می‌زند و بعد `UPDATE`؛ زیر بار هم‌زمان پول گم می‌شود.
- ستون `id` را `SERIAL` (۳۲ بیتی) می‌سازد؛ آن‌وقت `GET /accounts/999999999999` به‌جای `404` خطای `500` می‌گیرد.
- SQL را با `fmt.Sprintf` می‌سازد (خطر SQL injection).
- برای `POST` به‌جای `201` همان `200` را برمی‌گرداند.
- طول نام را با `len()` می‌شمارد که بایت است، نه rune؛ پس نام‌های فارسی را اشتباه رد می‌کند.
- `Content-Type` را فقط برای پاسخ‌های موفق تنظیم می‌کند.

🚩 **نقطه‌ی بازرسی ۱:** چند کامیت کوچک (`feat: domain types` ، `feat: memory store` ، `feat: http api` ، `feat: postgres store`) و بعد merge در `main` .

مرحله‌ی ۲ — تست‌ها

⏱️ ۱:۰۵ تا ۱:۲۰ · 💯 ۸ امتیاز · 🌿 `test/unit`

🎯 ثابت کن کدت درست کار می‌کند، **بدون نیاز به دیتابیس**.

✅ **کارها:**

- [ ] تست جدول‌محور (table-driven) برای اعتبارسنجی، یا برای `Withdraw` / `Transfer` روی store درون‌حافظه‌ای
- [ ] تست هم‌زمانی: ۱۰۰ goroutine به‌همراه `sync.WaitGroup` روی store درون‌حافظه‌ای
- [ ] تست handlerها با `net/http/httptest`
- [ ] اجرای `go test -race ./...`

🏁 **تمام است وقتی:** `go test -race -count=1 ./...` بدون هیچ دیتابیسی سبز است.

💯 **امتیازها:**

- [ ] **(۳)** تست جدول‌محور با `t.Run` و دست‌کم ۵ حالت، شامل حالت‌های خطا که با `errors.Is` بررسی می‌شوند
- [ ] **(۲)** تست هم‌زمانی روی store درون‌حافظه‌ای: دست‌کم ۱۰۰ goroutine و `sync.WaitGroup` ، بررسی دقیق نتیجه‌ی نهایی، و پاس شدن با `-race`
- [ ] **(۲)** تست `httptest` برای دست‌کم ۳ endpoint، که دست‌کم یکی از آن‌ها حالت خطا باشد
- [ ] **(۱)** `go test -race ./...` بدون اجرای دیتابیس پاس می‌شود

💡 **اسکلت یک تست جدول‌محور:**

```
tests := []struct {
    name    string
    owner   string
    wantErr error
}{
    {"valid", "Sara", nil},
    {"persian name", "سارا", nil},
    {"too short", "a", bank.ErrInvalidOwner},
    {"only spaces", "   ", bank.ErrInvalidOwner},
    {"too long", strings.Repeat("x", 41), bank.ErrInvalidOwner},
}
for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        _, err := bank.NormalizeOwner(tt.owner)
        if !errors.Is(err, tt.wantErr) {
            t.Fatalf("got %v, want %v", err, tt.wantErr)
        }
    })
}
```

- تست یکپارچه با PostgreSQL اختیاری است. اگر نوشتی، وقتی متغیر محیطی مخصوصش تنظیم نشده بود آن را skip کن، مثلاً: `if os.Getenv("MINIBANK_TEST_DATABASE_URL") == "" { t.Skip("no database") }`

🤖 **تله‌های هوش مصنوعی:** تست‌هایی می‌نویسد که به دیتابیس واقعی وصل می‌شوند و در CI شکست می‌خورند؛ از `t.Parallel()` با state مشترک استفاده می‌کند؛ تست هم‌زمانی می‌نویسد ولی نتیجه‌ی دقیق را بررسی نمی‌کند.

🚩 **نقطه‌ی بازرسی ۲:** کامیت `test: table-driven, concurrency and http tests` و merge.

مرحله‌ی ۳ — لینوکس و اسکریپت‌های Bash

⏱️ ۱:۲۰ تا ۱:۳۵ · 💯 ۷ امتیاز · 🌿 `chore/scripts`

🎯 دو اسکریپت واقعی که هر مهندس DevOps هر روز می‌نویسد.

✅ **کارها:**

- [ ] `scripts/check-env.sh` : بررسی می‌کند همه‌ی ابزارها نصب و آماده باشند
- [ ] `scripts/smoke.sh [BASE_URL]` : یک تست دود سریع روی برنامه‌ی در حال اجرا (پیش‌فرض `http://localhost:8080`)
- [ ] `chmod +x scripts/*.sh`

🏁 **تمام است وقتی** خروجی تقریباً این شکلی باشد:

```
$ scripts/check-env.sh
✓ go go1.26.2
✓ git 2.47.0
✓ docker 27.3.1
✓ docker daemon reachable
✓ kubectl v1.31.0
✓ kind 0.24.0
ALL GOOD

$ scripts/smoke.sh http://localhost:8080
PASS healthz
PASS readyz
PASS create account A
PASS create account B
PASS deposit
PASS transfer
PASS overdraft is rejected (409)
PASS unknown account (404)
8 passed, 0 failed
```

💯 **امتیازها — هر دو اسکریپت:**

- [ ] (۱) در **هر دو اسکریپت** (`check-env.sh` و `smoke.sh`، هر کدام نیم امتیاز): خط اول دقیقاً `#!/usr/bin/env bash`، قبل از اولین دستور `set -euo pipefail`، بدون خطای `bash -n`، و بیت اجرایی (mode `100755`) کامیت شده باشد

💯 **امتیازها — `check-env.sh` :**

- [ ] **(۱)** وجود kubectl، docker، git، go و (kind یا minikube) را بررسی کند و برای هر کدام ✓ یا ✗ چاپ کند؛ اگر همه بودند کد خروج ۰ و در غیر این صورت ۱؛ خط آخر `ALL GOOD` یا `MISSING: …`
- [ ] **(۱)** روشن بودن Docker daemon را هم بررسی کند (`docker info`) و پیام واضح بدهد

💯 **امتیازها — `smoke.sh` :**

- [ ] **(۱)** دست‌کم ۶ بررسی با curl روی status code؛ پیشنهاد: healthz، readyz، ساخت دو حساب، واریز، انتقال، برداشت بیش از موجودی (باید `409` بدهد)، حساب ناموجود (باید `404` بدهد)
- [ ] **(۱)** برای هر بررسی یک خط چاپ کند که **با** `PASS <name>` یا `FAIL <name> (expected X, got Y)` **شروع شود** (بدون رنگ یا علامت قبل از آن)، خط آخر دقیقاً به شکل `N passed, M failed` باشد (مثلاً `8 passed, 0 failed`)، و اگر چیزی شکست خورد کد خروج غیرصفر بدهد. **حتی وقتی سرور اصلاً در دسترس نیست** باید خط‌های `FAIL` و جمع‌بندی چاپ شوند، نه این‌که اسکریپت بی‌صدا متوقف شود (تصحیح‌گر این را با یک پورت بسته امتحان می‌کند)
- [ ] **(۱)** شناسه‌ها را از پاسخ‌ها استخراج کند (هیچ id ثابتی در اسکریپت نباشد؛ فقط برای بررسی «حساب ناموجود» از یک id خیلی بزرگ مثل `999999999999` استفاده کن)
- [ ] **(۱)** روی Compose **و** روی port-forward کلاستر Kubernetes سبز باشد

💡 **نکته‌ها:**

```
code=$(curl -s -o /tmp/body.json -w '%{http_code}' --max-time 5 \
  -X POST "$BASE/accounts" -d '{"owner":"Smoke"}') || code=000

# با jq:
id=$(jq -r '.id // empty' /tmp/body.json 2>/dev/null || true)

# بدون jq:
id=$(grep -o '"id":[0-9]*' /tmp/body.json 2>/dev/null | cut -d: -f2 || true)
```

- ⚠️ با `set -e` ، اگر curl وصل نشود (کد خروج ۷) یا jq و grep چیزی پیدا نکنند، اسکریپت همان‌جا و **بدون هیچ خط FAIL** می‌میرد؛ برای همین `|| code=000` و `|| true` را بگذار.
- ⚠️ تله‌ی معروف Bash: با `set -e` ، دستور `((passed++))` وقتی `passed` صفر است کد خروج ۱ برمی‌گرداند و **اسکریپت را می‌کشد!** به‌جایش بنویس `passed=$((passed + 1))` .
- کاربران ویندوز: بیت اجرایی را با `git update-index --chmod=+x scripts/*.sh` ثبت کنید.
- با `command -v kind >/dev/null 2>&1` وجود یک ابزار را بدون خطا بررسی کن.

🚩 **نقطه‌ی بازرسی ۳:** کامیت `chore: env check and smoke test scripts` و merge.

مرحله‌ی ۴ — Docker

⏱️ ۱:۳۵ تا ۱:۵۵ · 💯 ۹ امتیاز · 🌿 `build/docker`

🎯 یک ایمیج **کوچک، امن و سریع** برای برنامه‌ات.

✅ **کارها:**

- [ ] یک `Dockerfile` چندمرحله‌ای (multi-stage)

- [ ] یک `.dockerignore`
- [ ] build و اجرای آزمایشی

🏁 تمام است وقتی:

```
docker build -t minibank:v1 .
docker images minibank:v1
docker run --rm -p 8080:8080 -e STORE=memory minibank:v1
```

اندازه‌ی ایمیج باید حداکثر ۳۰ مگابایت باشد. وقتی کانتینر بالا آمد، در یک ترمینال دیگر `curl -i localhost:8080/healthz` باید جواب `200` بدهد.

💯 امتیازها:

- [ ] **(۲)** build چندمرحله‌ای: مرحله‌ی ساخت با `golang` و مرحله‌ی نهایی `scratch`، distroless یا `alpine`
- [ ] **(۲)** اندازه‌ی ایمیج نهایی حداکثر ۳۰ مگابایت
- [ ] **(۱)** اجرا با کاربر غیر root (مثلاً `USER 65532:65532`)
- [ ] **(۱)** فایل `.dockerignore` (حذف `.git`، باینری‌ها، `.env`، `k8s/` و …)
- [ ] **(۱)** ترتیب مناسب برای cache: اول `COPY go.mod go.sum`، بعد `go mod download`، و بعد `COPY . .`
- [ ] **(۱)** باینری استاتیک با `CGO_ENABLED=0` و `-ldflags "-s -w"`؛ `ENTRYPOINT` به شکل exec (آرایه‌ای)؛ `EXPOSE 8080`
- [ ] **(۱)** `docker build` بدون خطا کار کند و کانتینر با `STORE=memory` روی `/healthz` جواب `200` بدهد

💡 اسکلت (بقیه‌اش با خودت):

```
# ---------- مرحله‌ی ساخت ----------
# تگ golang باید دست‌کم برابر خط go در go.mod باشد
FROM golang:1.26-alpine AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /out/minibank ./cmd/minibank

# ---------- مرحله‌ی نهایی ----------
FROM scratch
# USER، EXPOSE، ENTRYPOINT ،کپی باینری…
```

- در Dockerfile کامنت فقط در خطی جدا که با `#` شروع می‌شود مجاز است؛ `#` در انتهای یک دستور، جزو آرگومان‌های همان دستور حساب می‌شود و build را خراب می‌کند.
- تگ ایمیج `golang` باید **دست‌کم برابر** خط `go` در `go.mod` باشد. ایمیج‌های رسمی `GOTOOLCHAIN=local` دارند، پس ایمیج قدیمی‌تر build را با خطا متوقف می‌کند.
- ایمیج `scratch` هیچ shell و curlی ندارد؛ پس `docker exec … sh` کار نمی‌کند و `HEALTHCHECK` به یک فلگ در خود باینری نیاز دارد (مثلاً `-healthcheck`، مثل user-service در پروژه‌ی نهایی بوت‌کمپ). این کار اختیاری است.

🤖 **تله‌های هوش مصنوعی:** ایمیج نهایی را همان `golang` می‌گذارد (بیش از ۸۰۰ مگابایت)؛ کانتینر را با root اجرا می‌کند؛ `COPY . .` را قبل از `go mod download` می‌گذارد و cache را خراب می‌کند؛ از تگ `latest` استفاده می‌کند.

🚩 **نقطه‌ی بازرسی ۴:** کامیت `build: multi-stage dockerfile` و merge.

مرحله‌ی ۵ — Docker Compose

⏱️ ۱:۵۵ تا ۲:۱۰ · 💯 ۷ امتیاز · 🌿 `build/compose`

🎯 کل سیستم (برنامه + PostgreSQL) با یک دستور بالا بیاید.

✅ **کارها:**

- [ ] فایل `compose.yaml` با دو سرویس `app` و `db`
- [ ] فایل `.env.example` کامیت شود؛ فایل `.env` در `.gitignore` باشد
- [ ] اجرای smoke test روی Compose

🏁 **تمام است وقتی:**

```
cp .env.example .env
docker compose up -d --build
scripts/smoke.sh http://localhost:8080
docker compose down && docker compose up -d
curl -s localhost:8080/accounts
```

smoke test باید سبز باشد، و بعد از `down` و `up` (بدون `-v`) حساب‌ها هنوز باید باشند.

💯 **امتیازها:**

- [ ] **(۱)** دو سرویس `app` (ساخته‌شده از Dockerfile خودت) و `db` (`postgres:16-alpine`)
- [ ] **(۲)** healthcheck برای دیتابیس (`pg_isready`) و `depends_on` با `condition: service_healthy` برای برنامه
- [ ] **(۱)** یک named volume برای داده‌ی دیتابیس؛ داده بعد از `docker compose down` و `up` (بدون `-v`) باقی بماند
- [ ] **(۱)** پیکربندی از طریق `.env`؛ فایل `.env.example` کامیت شده باشد، `.env` در `.gitignore` باشد و هرگز کامیت نشود، و هیچ رمزی داخل کد Go نوشته نشده باشد
- [ ] **(۱)** برای `app`، `restart: unless-stopped`؛ پورت دیتابیس به host منتشر (publish) **نشود**
- [ ] **(۱)** smoke test روی `docker compose up -d --build` از یک clone تمیز (بعد از `cp .env.example .env`) سبز باشد

💡 **نکته‌ها:**

- داخل Compose، آدرس دیتابیس **نام سرویس** است، نه `localhost`: `postgres://minibank:${POSTGRES_PASSWORD}@db:5432/minibank?sslmode=disable`
- healthcheck دیتابیس: `pg_isready -U minibank -d minibank`
- دستور `docker compose down -v` volumeها را پاک می‌کند و داده از بین می‌رود.
- لاگ زنده: `docker compose logs -f app`

🤖 **تله‌های هوش مصنوعی:** `depends_on` را بدون شرط می‌نویسد (برنامه قبل از آماده شدن دیتابیس بالا می‌آید و مدام crash می‌کند)؛ پورت `5432` را منتشر می‌کند؛ رمز را مستقیم در `compose.yaml` می‌نویسد.

🚩 **نقطه‌ی بازرسی ۵:** کامیت `build: docker compose with postgres` و merge.

مرحله‌ی ۶ — CI با GitHub Actions

⏱️ ۲:۱۰ تا ۲:۲۵ · 💯 ۶ امتیاز · 🌿 `ci/github-actions`

🎯 هر push و هر Pull Request به‌طور خودکار بررسی شود.

✅ **کارها:**

- [ ] فایل `.github/workflows/ci.yml` با دو job به نام‌های `test` و `build`
- [ ] این شاخه را push کن و در GitHub یک **Pull Request** باز کن
- [ ] صبر کن CI سبز شود، PR را در GitHub با دکمه‌ی **Create a merge commit** ادغام کن (نه «Squash and merge» و نه «Rebase and merge»؛ این دو کامیت merge نمی‌سازند)، و بعد با `git switch main && git pull` شاخه‌ی `main` محلی را به‌روز کن

💯 **امتیازها:**

- [ ] **(۱)** اجرا روی push به `main` و روی `pull_request`
- [ ] **(۲)** job `test`: `actions/checkout`، `actions/setup-go` (با `go-version-file: go.mod`)، بررسی gofmt که در صورت مشکل build را fail کند، `go vet ./...` و `go test -race ./...`
- [ ] **(۱)** job `build`: اجرای `docker build` با `needs: test`
- [ ] **(۱)** آخرین اجرا روی `main` سبز است و badge آن در README هست
- [ ] **(۱)** دست‌کم یک مرحله از طریق Pull Request در GitHub ادغام شده و چک CI روی آن دیده می‌شود

💡 **نکته‌ها:**

- بررسی gofmt: `test -z "$(gofmt -l .)"`
- badge: `![CI](https://github.com/<user>/<repo>/actions/workflows/ci.yml/badge.svg)`
- چون تست‌هایت به دیتابیس نیاز ندارند، CI به سرویس PostgreSQL هم نیازی ندارد.

🤖 **تله‌های هوش مصنوعی:** از نسخه‌های قدیمی actionها استفاده می‌کند (مثلاً `@v2`)؛ تستی می‌نویسد که به دیتابیس نیاز دارد و CI قرمز می‌شود؛ `needs: test` را فراموش می‌کند.

🚩 **نقطه‌ی بازرسی ۶:** merge از طریق Pull Request در GitHub.

مرحله‌ی ۷ — Kubernetes

⏱️ ۲:۲۵ تا ۲:۵۰ · 💯 ۱۰ امتیاز · 🌿 `deploy/k8s`

🎯 برنامه با **دو رپلیکا** و PostgreSQL روی کلاستر محلی (kind یا minikube) اجرا شود.

✅ **فایل‌ها:**

```
k8s/
├── 00-namespace.yaml    # namespace: minibank
├── 10-configmap.yaml    # PORT، STORE=postgres
├── 11-secret.yaml       # POSTGRES_PASSWORD، DATABASE_URL
├── 20-postgres.yaml     # (emptyDir یا PVC +) Deployment + Service
└── 30-app.yaml          # (پورت ۸۰ به ۸۰۸۰) Service + با ۲ رپلیکا Deployment
```

✅ **دستورها:**

```
docker compose down
docker build -t minibank:v1 .
kind load docker-image minibank:v1 --name exam
kubectl apply -f k8s/
kubectl -n minibank get pods -w
```

- `docker compose down` پورت 8080 را آزاد می‌کند (بدون `-v`، پس داده‌ی Compose می‌ماند). در minikube به‌جای `kind load` بزن: `minikube image load minibank:v1`
- بعد از **هر** build دوباره `kind load` کن و `kubectl -n minibank rollout restart deploy/minibank` بزن؛ چون تگ همان `v1` است، podها خودشان ایمیج جدید را نمی‌گیرند.
- `get pods -w` را وقتی همه `Running` شدند با Ctrl+C ببند. بعد port-forward را در یک ترمینال جدا باز نگه دار و smoke test را در ترمینال دیگر بزن:

```
kubectl -n minibank port-forward svc/minibank 8080:80
```

```
scripts/smoke.sh http://localhost:8080
```

🏁 **تمام است وقتی:** دو pod برنامه در وضعیت `Running` و `READY 1/1` هستند و smoke test روی port-forward سبز است.

💯 **امتیازها:**

- ☐ **(۱)** Namespace `minibank`؛ و `kubectl apply -f k8s/` روی یک کلاستر تازه **در یک مرحله** کار کند
- ☐ **(۲)** ConfigMap برای تنظیمات غیرمحرمانه و Secret برای رمز یا رشته‌ی اتصال؛ برنامه آن‌ها را با `envFrom` یا `secretKeyRef` بخواند
- ☐ **(۲)** Deployment و Service (از نوع ClusterIP) برای PostgreSQL؛ با PVC یا emptyDir (و دلیل انتخابت را در README بنویس)
- ☐ **(۲)** Deployment برنامه: `replicas: 2`، هم `requests` و هم `limits` برای منابع، تگ ثابت ایمیج (نه `latest`)، و `imagePullPolicy: IfNotPresent`
- ☐ **(۲)** `livenessProbe` روی `/healthz` و `readinessProbe` روی `/readyz`
- ☐ **(۱)** Service برنامه به نام `minibank` (از نوع ClusterIP) با `port: 80` و `targetPort: 8080`، تا `kubectl -n minibank port-forward svc/minibank 8080:80` کار کند و smoke test روی آن سبز باشد

💡 **نکته‌ها:**

- `kubectl apply -f k8s/` فایل‌ها را به **ترتیب حروف الفبا** اعمال می‌کند؛ برای همین اسم فایل‌ها با عدد شروع می‌شود تا namespace اول ساخته شود.
- اگر تگ ایمیج `latest` باشد، Kubernetes پیش‌فرض `imagePullPolicy: Always` می‌گذارد، سعی می‌کند ایمیج را از Docker Hub بکشد و به `ErrImagePull` می‌خورد. پس `minibank:v1` و `IfNotPresent` .
- داخل کلاستر، آدرس دیتابیس **نام Service** آن است: `postgres` (یا `postgres.minibank.svc.cluster.local`).
- در Secret می‌توانی با `stringData` متن ساده بنویسی. یادت باشد base64 **رمزنگاری نیست**. در پروژه‌ی واقعی Secret را کامیت نمی‌کنیم؛ اینجا مجاز است، ولی یک کامنت درباره‌اش بنویس.
- **liveness را روی `/readyz` نگذار!** اگر دیتابیس قطع شود، Kubernetes همه‌ی podهای سالم برنامه را بی‌دلیل restart می‌کند.
- اگر برنامه‌ات **قبل از** بالا آوردن HTTP منتظر اتصال به Postgres می‌ماند، در اولین `kubectl apply` ممکن است liveness آن را پیش از آماده شدن دیتابیس بکشد (`CrashLoopBackOff`). راه حل: یک `startupProbe` روی `/healthz` با `periodSeconds: 2` و `failureThreshold: 30` بگذار، یا اول HTTP را بالا بیاور و بگذار `/readyz` تا وصل شدن دیتابیس `503` بدهد.
- timeout پیش‌فرض probeها ۱ ثانیه است، ولی `/readyz` ممکن است تا ۲ ثانیه منتظر ping بماند. برای readiness `timeoutSeconds: 3` بگذار.
- اگر pod دیتابیس روی `ImagePullBackOff` ماند (Docker Hub در دسترس نیست)، ایمیج را از Docker خودت داخل کلاستر بارگذاری کن: `kind load docker-image postgres:16-alpine --name exam`
- **ابزارهای رفع اشکال:**

```
kubectl -n minibank get pods
kubectl -n minibank describe pod <name>
kubectl -n minibank logs deploy/minibank
kubectl -n minibank get events --sort-by=.lastTimestamp
```

🤖 **تله‌های هوش مصنوعی:** `image: minibank:latest` ؛ بدون محدودیت منابع؛ liveness روی `/readyz` ؛ رمز دیتابیس به‌صورت متن ساده در خود Deployment.

🚩 **نقطه‌ی بازرسی ۷:** کامیت `deploy: kubernetes manifests` و merge.

مرحله‌ی ۸ — مستندات، یادداشت AI و تحویل

⏱️ ۲:۵۰ تا ۳:۰۰ · 💯 ۳ امتیاز · 🌿 `docs/readme`

🎯 کسی که ریپازیتوری‌ات را باز می‌کند، در ۲ دقیقه بفهمد این چیست و چطور اجرایش کند.

💡 **اسکلت README خودت:**

```
# MiniBank
![CI](https://github.com/<user>/<repo>/actions/workflows/ci.yml/badge.svg)

یک خط معرفی.

## Architecture
(نمودار ASCII یا Mermaid)

## Run
### Local (STORE=memory)
### Docker Compose
### Kubernetes (kind)

## Configuration
(جدول متغیرهای محیطی)

## AI usage
- ابزارها:
- برای چه کاری:
- اشتباه‌هایی که AI کرد و من درستشان کردم:
```

💯 امتیازها:

- ☐ **(۱)** README: معرفی، نمودار معماری (ASCII یا Mermaid)، نحوه‌ی اجرا (memory، Compose، Kubernetes) و جدول متغیرهای محیطی
- ☐ **(۱)** بخش AI usage: ابزارها، کاربردشان، و دست‌کم یک اشتباه AI که خودت درستش کردی
- ☐ **(۱)** تاریخچه‌ی Git: دست‌کم ۱۰ کامیت معنادار (به‌جز کامیت‌های merge) که در طول آزمون پخش شده باشند (فاصله‌ی اولین تا آخرین کامیت دست‌کم ۹۰ دقیقه)، پیام‌ها به سبک conventional (`feat:`، `fix:`، `test:`، `build:`، `ci:`، `docs:`، `chore:`)، و هیچ رمز واقعی یا باینری در ریپازیتوری (Secret نمایشی Kubernetes با کامنت مجاز است)

✅ چک‌لیست ۵ دقیقه‌ی آخر:

- ☐ همه‌ی شاخه‌ها در `main` ادغام و push شده‌اند
- ☐ CI روی `main` سبز است
- ☐ برنامه در Kubernetes در حال اجراست و port-forward روی `8080` برای تصحیح‌گر آماده است
- ☐ جدول خودارزیابی (پایین همین فایل) پر شده است

⭐ امتیازهای اضافه (توصیه: بعد از تمام شدن مراحل اصلی) · حداکثر +۱۵

کد	چالش	امتیاز
B1	**متریک‌های Prometheus** روی `/metrics`: شمارنده‌ی `http_requests_total` با برچسب‌های `method`، `path` و `status` (مقدار `path` الگوی مسیر باشد مثل `/accounts/{id}`، نه `/accounts/17`)، هیستوگرام `http_request_duration_seconds`، و شمارنده‌ی `minibank_transfers_total` با برچسب `result` (یکی از `ok`، `insufficient_funds`، `error`)	+۴
B2	**Ingress** (با ingress-nginx): میزبان `minibank.local` به Service برنامه برود، و درخواست واقعاً از کنترلر ingress-nginx رد شود (فقط وجود شیء Ingress کافی نیست؛ راهنمایی B2 را ببین)	+۳

کد	چالش	امتیاز
B3	**چارت Helm** در `charts/minibank/` با valueهای `image.tag`، `replicaCount` و `resources` که **کل سیستم** را می‌سازد (برنامه، ConfigMap، PostgreSQL و Secret)؛ دستور `helm install minibank charts/minibank -n minibank-helm --create-namespace` کار کند	۴+
B4	**انتقال ضد بن‌بست:** تصحیح‌گر ۲۰۰ انتقال هم‌زمان رفت‌وبرگشتی (از حساب P به Q و از Q به P) می‌فرستد. همه باید `201` باشند و پول دقیقاً حفظ شود	۴+

💡 **راهنمایی B1:** چرا `path` باید الگوی مسیر باشد؟ چون اگر هر id یک مقدار جدید برچسب بسازد، تعداد سری‌های زمانی بی‌نهایت می‌شود (مشکل cardinality). از Go 1.23 به بعد، ServeMux الگوی مسیر را در `r.Pattern` می‌گذارد، ولی **فقط موقع مسیریابی**: در middleware آن را **بعد از** `next.ServeHTTP(w, r)` بخوان (قبلش خالی است). مقدارش متد را هم دارد (مثلاً `GET /accounts/{id}`) که قبول است؛ برای درخواستی که به هیچ الگویی نخورده خالی است، پس آن را `unmatched` بگذار، نه `r.URL.Path` . مسیر `/metrics` را هم با متد ثبت کن: `GET /metrics` . (در Go 1.22 از `mux.Handler(r)` استفاده کن.)

💡 **راهنمایی B3:** چارت را در یک namespace جدا (`minibank-helm`) نصب کن تا با منابعی که در S7 با `kubectl apply -f k8s/` ساخته‌ای تداخل نداشته باشد (اگر در همان namespace نصب کنی، Helm با خطای `invalid ownership metadata` متوقف می‌شود). استقرار S7 را پاک نکن؛ تصحیح روی همان انجام می‌شود.

💡 **راهنمایی B4:** اگر دو تراکنش هم‌زمان ردیف‌ها را به ترتیب متفاوت قفل کنند (یکی اول P بعد Q و دیگری اول Q بعد P)، PostgreSQL خطای deadlock با کد `40P01` می‌دهد و کاربر `500` می‌بیند. از خودت بپرس: چطور می‌شود ترتیب قفل کردن را برای همه‌ی تراکنش‌ها یکسان کرد، و اگر با این حال `40P01` آمد، برنامه چه باید بکند؟ (اگر پول حفظ شود ولی بعضی پاسخ‌ها خطا باشند، فقط ۱ از ۴ امتیاز می‌گیری.)

💡 **راهنمایی B2 در kind:** کلاستر `exam` را پاک نکن؛ لازم نیست از نو بسازی. این دستورها روی کلاستر معمولی kind امتحان شده‌اند (ایمیج‌های ingress-nginx از `registry.k8s.io` می‌آیند؛ اگر در ایران دانلود نشدند، این امتیاز را رها کن):

```
kubectl label node exam-control-plane ingress-ready=true
kubectl apply -f https://kind.sigs.k8s.io/examples/ingress/deploy-ingress-nginx.yaml
kubectl -n ingress-nginx wait --for=condition=ready pod -l app.kubernetes.io/component=controller
--timeout=180s
kubectl -n ingress-nginx port-forward svc/ingress-nginx-controller 8081:80
```

و در ترمینال دیگر: `curl -i -H 'Host: minibank.local' http://localhost:8081/healthz` باید `200` بدهد. در Ingress خودت `ingressClassName: nginx` بگذار.

🎤 دفاع حضوری — چه انتظاری داشته باش (۲۰ امتیاز، بدون هوش مصنوعی)

بخش	چه اتفاقی می‌افتد	امتیاز
D1	مسیر کامل یک درخواست را در کد خودت نشان می‌دهی: از router به handler، از handler به store، از store به دیتابیس، و در آخر پاسخ	۵
D2	یک سؤال «چرا؟» درباره‌ی یکی از تصمیم‌هایت	۵
D3	مدرس چیزی را در استقرار تو خراب می‌کند؛ حدود ۳ دقیقه وقت داری که با لاگ‌ها و `kubectl` مشکل را پیدا و درست کنی	۵
D4	یک تغییر کوچک زنده در کمتر از ۳ دقیقه، بدون هوش مصنوعی	۵

مقیاس هر بخش: ۵ = واضح و درست · ۳ = نسبی · ۱ = مبهم · ۰ = نمی‌داند

قوانین دفاع: حدود ۷ تا ۱۲ دقیقه روی لپ‌تاپ خودت. مستندات رسمی (مثل pkg.go.dev و kubernetes.io) مجاز است، ولی هیچ ابزار هوش مصنوعی (حتی Copilot و افزونه‌های ادیتور) مجاز نیست؛ قبل از دفاع آن‌ها را خاموش کن.

برای تمرین، قبل از دفاع به این‌ها برای خودت جواب بده:

- مسیر `POST /transfers` را از router تا دیتابیس در کد خودت دنبال کن.
- اگر PostgreSQL قطع شود، `/healthz` و `/readyz` چه جوابی می‌دهند و Kubernetes با هر کدام چه می‌کند؟
- چرا با `STORE=memory` نمی‌شود دو رپلیکا داشت؟
- یک pod در وضعیت `CrashLoopBackOff` است. قدم‌به‌قدم چه می‌کنی؟

> ⚠️ **قانون «توضیح بده یا از دست بده»:** اگر بعد از دو سؤال ساده‌تر نتوانی یک مورد امتیازی (یک چک‌باکس همین فایل) را توضیح بدهی، امتیاز همان مورد صفر می‌شود؛ چه امتیاز ساخت باشد و چه امتیاز اضافه.

📊 جدول خودارزیابی (قبل از تحویل پر کن)

مرحله	حداکثر	تخمین من	نمره‌ی مدرس
S0 شروع و تصمیم‌ها	۵		
S1 برنامه‌ی Go	۲۵		
S2 تست‌ها	۸		
S3 لینوکس و Bash	۷		
S4 Docker	۹		
S5 Docker Compose	۷		
S6 CI	۶		
S7 Kubernetes	۱۰		
S8 مستندات و تاریخچه	۳		
جمع ساخت	**۸۰**		
D دفاع حضوری	۲۰	—	
امتیاز اضافه (B1 تا B4)	۱۵		
نمره‌ی نهایی	**۱۰۰ (+۱۵)**		

نمره‌ی قبولی: ۶۰ (نمره‌ی نهایی به نزدیک‌ترین عدد صحیح گرد می‌شود). موفق باشی! 🐹